**Parcial Algoritmos y Programación II - 7541  Curso Calvo  (2º instancia)**  21 de junio de 2019

**Padrón:**  **Apellido y Nombre:**

1) Indicar la salida por pantalla y escribir las sentencias necesarias para liberar correctamente la memoria.

```
int main(){
  int *A, *C, *F;
  int **B;
  char *D, *E;
  int H = 66;
  char G = 'C';
  F = new int[3];
  for (int i = 0; i < 3; i++) {
      F[i] = H + i;
  }
  H++;
  A = F;
  C = new int;
  (*C) = A[1];
  A = F + 2;
  cout << (*F) << (*C) << A[0] << endl;
  B = &C;
  A = C;
  C = F + 1;
  (**B) = H + 2;
  cout <<  (*C) <<  (**B) << (*A) << endl;
  D = (char*) A;
  E = (char*) *B;
  F[1] = 70;
  cout << (*D) << (*E) << (*A) << endl;
  E = &G;
  H++;
  G = (char) H;
  H++;
  (*A) = H;
  cout << (*E) << (*D) << G << endl;

  // liberar la memoria
  // ...
  return 0;
}
```

**2.** Implementar para la clase Lista<T> con una estructura **doblemente** enlazada el siguiente método, indicando pre y post:

```
T remover(unsigned int posicionElemento);
```

**3.** Implementar el método `buscarRecetaSustituta` de la clase `Chef` a partir de las siguientes especificaciones:

```
class Chef {
  public:
    /* post: busca en la lista recetasAlternativas una Receta que pueda ser preparada con los mismos Ingredientes que
     *       recetaPorSustituir. Considera que todos los Ingredientes de recetaPorSustituir sean Ingredientes de
     *       la otra Receta comparando su nombre y validando que la cantidad sea inferior.
     *       Devuelve la Receta que cumpla la condición.
     */
    Receta* buscarRecetaSustituta(Receta* recetaPorSustituir, Lista<Receta*>* recetasAlternativas);
};
```

```
class Receta {

  public:

    /* post: inicializa la Receta sin Ingredientes asignados.
     */
    Receta(string nombre);

    /* post: elimina todos los Ingrediente de la Receta.
     */
    ~Receta();

    /* post: devuelve todos los Ingredientes de la Receta.
     */
    Lista<Ingrediente*>* getIngredientes();

    /* post: devuelve el nombre que identifica a la Receta.
     */
    string getNombre();
};
```

```
class Ingrediente {

  public:

    /* post: queda inicializado con el nombre y
     *       cantidad indicados.
     */
    Ingrediente(string nombre, float cantidad);

    ~Ingrediente();

    /* post: devuelve el nombre del Ingrediente.
     */
    string getNombre();

    /* post: devuelve la cantidad del Ingrediente.
     */
    float getCantidad();
};
```

**4.** Diseñar la especificación e implementar el **TDA Escudo**. Debe proveer operaciones para:

- crearlo recibiendo como parámetro los puntos de resistencia que tiene disponibles y la cantidad máxima de reparaciones permitidas.
- defenderAtaque:  recibiendo como parámetro la cantidad de puntos del ataque, consume puntos de resistencia.
- estaDestruido: indica si el Escudo ya agotó sus puntos de resistencia.
- reparar: restaura la resistencia original, solamente si aún no fue destruido previamente.
- contarAtaquesSoportados: devuelve la cantidad de ataques que pudo defender sin ser destruido.

**Para aprobar es necesario tener al menos el 60% de cada uno de los ejercicios correctos y completos.**
**Para cada método escribir  pre y post condición, si recibe argumentos y cuáles, y si retorna un dato y cuál. De faltar ésto, se considerará el código incompleto.**
**Duración del examen : 3 horas**

```
666768
696967
CF67
DED
```

```
delete[] F;
delete A;
```

**Para aprobar es necesario tener al menos el 60% de cada uno de los ejercicios correctos y completos.**
**Para cada método escribir  pre y post condición, si recibe argumentos y cuáles, y si retorna un dato y cuál. De faltar ésto, se considerará el código incompleto.**
**Duración del examen : 3 horas**